

PRIMER LLIURAMENT PROP

SUBGRUP 41.2

Versió 0

Abel Fernández Leiva (abel.fernandez.leiva)

Judit Giner Velazquez (judit.giner)

Aura Han Ruiz Sánchez (aura.han.ruiz)

Paula Torreblanca Costa (paula.torreblanca)

INDEX

ESTRUCTURES DE DADES.....	2
FUNCIONAMENT DEL ALGORISME KMEANS.....	3
FUNCIONAMENT DE SILHOUETTE.....	6
MÈTODES I ATRIBUTS.....	7
DESCRIPCIÓ DE JOCS DE PROVES (DRIVERS).....	21
DESCRIPCIÓ DE CASOS D'ÚS.....	25
DIAGRAMA CASOS D'ÚS.....	28
DIAGRAMA DE CLASSES.....	29
CLASSES IMPLEMENTADES PER CADASCÚ.....	30

ESTRUCTURES DE DADES

1. `Object[][] data`.

Consisteix en un array d'objectes on es guarden totes les respostes d'una enquesta, després de processar-les (eliminar valors nulls i outliers), per tal d'enviar al algoritme KMeans, que s'encarrega de definir els clusters..

La tria d'aquesta estructura es basa en dos motius principalment. Per una banda, necessitàvem una estructura capaç de tractar qualsevol tipus de dada, des de Double fins a String. Definir les dades com a `Object` ens permet mantenir-les totes en una mateixa estructura, la qual cosa redueix la complexitat del codi. Per altra banda, hem triat un `Array` en comptes d'una altra estructura per tal de facilitar l'accés a dades. Poder accedir mitjançant índex permet poder iterar per tots els valors d'una manera senzilla, que era el nostre principal objectiu.

2. `int[] variableType`.

És un array que guarda, per a cada columna relevant (que correspon a una pregunta) en una enquesta, l'índex que identifica el tipus de pregunta de la que es tracta. La raó per la qual hem triat `Integers` en comptes d'un altre tipus de variable és per simplificar el programa, ja que utilitzar `Strings` seria més exhaustiu. Hem triat els índexs manualment de la següent manera:

0 → tipus numèric

1 → tipus qualitativa ordenada, resposta única (en aquest cas la resposta consisteix en un array de dos elements: l'índex de la opció triada, i el número d'opcions que té la pregunta, les quals necessitem pels càlculs de distàncies).

2 → tipus qualitativa no ordenada, resposta única

3 → tipus qualitativa no ordenada, resposta múltiple

4 → tipus text lliure

FUNCIONAMENT DEL ALGORISME KMEANS

Quan es crea una nova instància de KMeans, cal enviar un valor per la k (nombre de clusters) i un nombre màxim d'iteracions que podrà fer l'algoritme. Un cop fet això, s'envia l'array de dades de l'enquesta (ja preprocessades anteriorment) i un array que conté el tipus de variable de cada columna.

A continuació es criden les funcions per calcular el màxim i mínim de cada columna, però en realitat aquest càlcul només es realitza per columnes de tipus numèric, posant un -1 altrament. Això servirà més endavant per a poder normalitzar les distàncies que es calculin, evitant haver de calcular aquests valors per a cada parella de dades.

El següent és una primera inicialització dels centroides dels clusters (k en total) que es fa seguint la lògica de KMeans++. En aquest algoritme, es tria el primer centroide de manera aleatòria, que serà el nostre valor de referència. Per la resta de centroides, la idea és trobar aquells més llunyans a aquest primer, per tal de tenir un bon punt de partida. Per fer-ho, calculem la distància mínima de cada punt al centroide actual amb el mètode `calculateMinDistances`, que ens retorna un vector amb la distància mínima entre cada punt i el centroide. A partir d'això, elevem al quadrat totes les distàncies (per obtenir valors més significatius amb els que treballar) i les sumem en una mateixa variable, que correspondrà al total (i que ens serveix per normalitzar els valors). Llavors, es calcula la probabilitat acumulada per cada punt (els punts amb distàncies més grans des del centroide inicial tenen una probabilitat major) i, finalment, es tria un valor entre 0-1 aleatòriament per tal d'escollir en quina part de la probabilitat acumulada ens trobem. Aquesta elecció correspon a un punt del dataset, que serà triat com a centroide. A partir d'aquest centroide, es repeteix el procés fins a tenir els k centroides que es volen.

Un cop feta la selecció dels centroides inicials, passem a la implementació del KMeans. Hi ha un bucle que farà, com a màxim, `maxIterations` (atribut definit al principi). Això garanteix que l'algoritme finalitza en qualsevol cas, inclús si el clustering no és l'ideal. A partir d'aquí, s'itera per totes les files del array que conté les respostes, assignant cadascuna al cluster amb centroide més proper. Per fer-ho, mira les distàncies de tots els centroides al punt actual, i l'assigna al més proper. Per calcular la distància s'utilitza la classe `CalculateDistance`, que explicarem més endavant. Un cop s'assigna cada punt al cluster que li correspon, es recalculen els centroides per obtenir valors representatius de cada cluster. En aquest cas, els centroides ja no seran necessàriament punts existents en les dades, sinó que seran punts representatius del cluster. Aquest càlcul es realitza, per cada cluster identificat, de la següent manera, segons el tipus de pregunta (especificats a dalt), i es

guarda a un `Object[]` reference, que servirà per representar aquest centroid, amb valors per a cada columna com si fos un punt més de les dades:

- 0 → càlcul de la mitjana de tots els elements del cluster.
- 1 → càlcul de la mitjana de tots els elements del cluster, el número d'opcions es manté constant.
- 2 → càlcul de la moda entre tots els punts del cluster, per trobar quin és el més repetit.
- 3 → es guarda en un Set (per evitar repeticions) el nombre de vegades que apareix cada resposta, i un cop fet això es selecciona aquelles que apareixen almenys en la meitat de les respostes.
- 4 → càlcul, per a cada element del cluster, de la distància Levenshtein amb tota la resta d'elements al cluster. S'escolleix reference com l'element amb menor distància acumulada amb la resta de punts.

A tot això, tenim un booleà, `changed`, que ens serveix per veure si l'algoritme ha convergit en cada iteració, és a dir, si no hi ha hagut cap canvi, la qual cosa indicaria que cada punt ja té la seva assignació definitiva i per tant no cal continuar iterant.

Finalment, queda detallar la lògica per, donats dos punts del dataset, calcular la distància entre ells. Aquesta distància és la suma de la distància entre els dos elements de cada columna, que es calcula de forma different depenent del tipus de columna:

- 0 → es calcula la distància entre els dos punts. Utilitzem els arrays `max` i `min` (calculats a la classe `KMeans`) per a normalitzar la dada, i elevem al quadrat per tal de que les distàncies siguin més significatives.
- 1 → es calcula la distància normalitzada entre els índexs de les dues opcions, ja que al estar ordenades, volem veure com de properes són.
- 2 → en el cas que les respostes siguin iguals, el valor és 0, altrament, és 1 (màxima distància).
- 3 → es calcula la distància de Jaccard: es guarda el nombre de respostes comunes entre els dos punts a la variable `intersection`, i després es calcula el nombre d'elements no compartits a `union`. Amb aquestes dades, es calcula la distància utilitzant la fórmula esmentada al document de l'assignatura.
- 4 → es calcula la distància Levenshtein normalitzada entre els dos strings. Aquesta distància representa el nombre de modificacions, insercions i eliminacions fetes per transformar un string a un altre. Aquest càlcul es fa utilitzant una lògica recursiva, que

per cada caràcter dins del string fa una comprovació. Si els caràcters són iguals, es passa al següent caràcter; sinó, es fan tres crides recursives que representen els tres canvis que es poden fer (explicats al principi). Amb això, es calcula quina d'aquestes tres crides retorna la distància més petita, amb la qual cosa obtenim la distància més curta.

Com es pot veure, totes les distàncies han estat normalitzades independentment del tipus de pregunta, amb l'objectiu de tractar de dades similars i evitar que unes columnes impactin més que d'altres en l'ànàlisi. Dist és una variable que emmagatzema la suma de distàncies de cada columna. Abans de fer el return, es divideix entre n (nombre de columnes) amb l'objectiu de retornar una distància mitjana, normalitzada.

El resultat que retorna la nostra implementació de KMeans és un Array on cada posició representa una resposta d'enquesta. Cada valor d'aquest array és un int entre 0 i k-1 que correspondrà al cluster assignat a aquella resposta.

FUNCIONAMENT DE SILHOUETTE

La mètrica de silhouette serveix per calcular la qualitat dels clusters generats pel K-means comparant la similitud entre els elements d'aquests. Per al seu càlcul s'utilitza la fórmula següent:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

On per un punt i:

- $b(i)$ és la distància mitjana mínima entre i i els elements d'un altre cluster
- $a(i)$ és la distància entre i i els elements del seu propi cluster
- $s(i)$ és l'indicador de qualitat del punt i

Una vegada aquesta fórmula s'ha aplicat per a tots els punts es treu la qualitat general del clustering calculant la mitjana de tots els punts de la següent manera:

$$S = \frac{1}{n} \sum_{i=1}^n s(i)$$

On:

- n és el nombre de punts totals
- S és la qualitat mitjana del clustering (resultat final silhouette)

Utilitzant aquesta fórmula hem pogut evaluar la qualitat de diverses instàncies de K-means i així decidir amb quina quedar-nos.

MÈTODES I ATRIBUTS

KMeans.java

1. `assignToCluster`: mètode encarregat d'assignar cada resposta a un cluster.

Paràmetres d'entrada:

- `Object[][] data` → array que guarda les respostes que s'utilitzen per l'anàlisi.
- `int[] variableType` → array amb el tipus de pregunta de cada columna.

Valor de sortida:

- `int[]` → retorna, per cada resposta, l'index del cluster al que pertany.

2. `initializeRandomCentroids`: inicialitza, utilitzant l'algorisme de KMeans++, els centroides inicials per a executar KMeans.

Paràmetres d'entrada:

- `Object[][] data` → array que guarda les respostes que s'utilitzen per l'anàlisi.
- `int n` → nombre de respostes per a l'enquesta a analitzar.
- `int[] variableType` → array amb el tipus de pregunta de cada columna.
- `double[] max` → per a les columnes de tipus numèric, indica el màxim valor.
- `double[] min` → per a les columnes de tipus numèric, indica el mínim valor.

Valor de sortida:

- `Object[][]` → retorna els centroides inicials que ha indicat KMeans++ (com a files senceres).

3. `findNearestCentroid`: troba el centroide del cluster més proper a una fila indicada.

Paràmetres d'entrada:

- `int i` → indica l'index de la fila que es vol analitzar.
- `int[] variableType` → array amb el tipus de pregunta de cada columna.
- `double[] max` → per a les columnes de tipus numèric, indica el màxim valor.
- `double[] min` → per a les columnes de tipus numèric, indica el mínim valor.
- `Object[][] data` → array que guarda les respostes que s'utilitzen per l'anàlisi.

Valor de sortida:

- `int` → retorna l'index del cluster al que pertany la dada determinada.

4. `calculateCentroids`: recalcula els centroides de cada cluster després de cada iteració.

Paràmetres d'entrada:

- `Object[][] data` → array que guarda les respostes que s'utilitzen per l'anàlisi.

- int n → nombre de respostes per a l'enquesta a analitzar.
- int[] variableType → array amb el tipus de pregunta de cada columna.

5. calculateCentroid: retorna, per un cluster, un centroide que millor representa al cluster.

Paràmetres d'entrada:

- List<Integer> clusterIndexes → llista amb els índexs de totes les dades que pertanyen a un determinat cluster.
- int[] variableType → array amb el tipus de pregunta de cada columna.
- Object[][] data → array que guarda les respostes que s'utilitzen per l'anàlisi.

Valor de sortida:

- Object[][] → retorna el centroide que millor representa un cluster.

KMeansPlusPlus.java

1. initializeCentroids: mètode encarregat d'inicialitzar els centroides dels clústers utilitzant l'algoritme KMeans++.

Paràmetres d'entrada:

- Object[][] data → array que guarda les respostes que s'utilitzen per l'anàlisi.
- int k → nombre de clusters que hi ha.
- int[] variableType → array amb el tipus de pregunta de cada columna.
- double[] max → per a les columnes de tipus numèric, indica el màxim valor.
- double[] min → per a les columnes de tipus numèric, indica el mínim valor.

Valor de sortida:

- int[] → retorna els centroides inicials trobats.

2. calculateMinDistances: calcula la distància mínima entre un punt i el seu centroide més proper.

Paràmetres d'entrada:

- Object[][] data → array que guarda les respostes que s'utilitzen per l'anàlisi.
- Object[][] centroids → array amb tots els centroides trobats fins el moment.
- int centroidsCount → nombre de centroides trobats fins al moment.
- int[] variableType → array amb el tipus de pregunta de cada columna.
- double[] max → per a les columnes de tipus numèric, indica el màxim valor.

- double[] min → per a les columnes de tipus numèric, indica el mínim valor.

Valor de sortida:

- double[] → retorna la distància per cada dada al seu centroid més proper.

3. chooseNextCentroid: calcula el següent centroid en basa a la probabilitat elevada al quadrat de les distàncies.

Paràmetres d'entrada:

- double[] distances → array amb les mínimes distàncies de cada dada a un centroid.
- Random random → element random que ens dona una probabilitat aleatòria.

Valor de sortida:

- int → retorna l'index del centroid trobat.

CalculateDistance.java

1. compute → retorna la distància normalitzada entre dues dades.

Paràmetres d'entrada:

- Object[] point → respostes d'una fila.
- Object[] centroid → centroid, respostes d'una altra fila.
- int[] variableType → array amb el tipus de pregunta de cada columna.
- double[] max → per a les columnes de tipus numèric, indica el màxim valor.
- double[] min → per a les columnes de tipus numèric, indica el mínim valor.

Valor de sortida:

- double → retorna la distància entre els dos punts.

2. numericDistance: calcula la diferència entre dos valors. Aquesta diferència és posteriorment normalitzada i s'eleva al quadrat per incrementar les diferències.

Paràmetres d'entrada:

- Object point → resposta numèrica.
- Object centroid → centroid, resposta numèrica de referència.
- double max → màxim de la columna a la que pertanyen les dades.
- double min → mínim de la columna a la que pertanyen les dades.

Valor de sortida:

- double → retorna la distància entre els dos punts.

3. `singleChoiceOrdered`: calcula la distància entre els índexs de dues respostes (ordenades) i la normalitza.

Paràmetres d'entrada:

- `Object point` → resposta única, ordenada.
- `Object centroid` → centroide, resposta única, ordenada, de referència.

Valor de sortida:

- `double` → retorna la distància entre els dos punts.

4. `singleChoiceUnordered`: calcula si dos respostes són iguals o no.

Paràmetres d'entrada:

- `Object point` → resposta única.
- `Object centroid` → centroide, resposta única de referència.

Valor de sortida:

- `double` → retorna 1 si la resposta no és la mateixa, 0 altrament.

5. `multipleChoiceUnordered`: calcula la semblança entre dos respostes multiresposta.

Paràmetres d'entrada:

- `Object point` → resposta múltiple.
- `Object centroid` → centroide, resposta múltiple de referència.

Valor de sortida:

- `double` → retorna la semblança normalitzada entre dos respostes.

6. `freeText`: calcula la semblança entre dos respostes de text lliure.

Paràmetres d'entrada:

- `Object point` → resposta de text lliure.
- `Object centroid` → centroide, resposta de text lliure de referència.

Valor de sortida:

- `double` → retorna la semblança normalitzada entre dos respostes.

KMeansDriver.java

1. `main`:

Paràmetres d'entrada:

- `String[] args` → qualsevol paràmetre que l'usuari pugui utilitzar en l'execució.

TestAccuracy.java

1. testAccuracyIris: calcula la precisió del clustering per a les dades de iris.csv

Paràmetres d'entrada:

- int[] clusters → array amb l'index del cluster al que pertany cada fila de les dades.
- List<Object> labels → llista amb la classificació real de cada fila de les dades.

2. testAccuracyLoanLoader: calcula la precisió del clustering per a les dades de loan-loader.csv

Paràmetres d'entrada:

- int[] clusters → array amb l'index del cluster al que pertany cada fila de les dades.
- List<Object> labels → llista amb la classificació real de cada fila de les dades.

AuxiliarMethods.java

1. levenshtein: calcula la distància de levenshtein entre dos strings.

Paràmetres d'entrada:

- String s1 → primer string.
- String s2 → segon string.

Valor de sortida:

- int → retorna la distància levenshtein entre dos strings.

2. calculateMode: calcula la moda per una llista d'elements de qualsevol tipus.

Paràmetres d'entrada:

- List<T> choices → llistat d'elements.

Valor de sortida:

- T → retorna l'element més freqüent en la llista.

3. calculateAverage: calcula la mitjana dels elements d'una llista.

Paràmetres d'entrada:

- List<Double> data → llistat d'elements.

Valor de sortida:

- Double → retorna la mitjana de tots els valors de la llista.

4. `calculateMinVector`: calcula, per a les columnes numèriques, el mínim valor de la columna.

Paràmetres d'entrada:

- `Object[][] data` → array que guarda les respostes que s'utilitzen per l'anàlisi.
- `int[] variableType` → array amb el tipus de pregunta de cada columna.

Valor de sortida:

- `double[]` → retorna un vector on les posicions que corresponen a columnes numèriques es guarda el mínim de la columna, altrament es guarda un -1.

5. `calculateMaxVector`: calcula, per a les columnes numèriques, el màxim valor de la columna.

Paràmetres d'entrada:

- `Object[][] data` → array que guarda les respostes que s'utilitzen per l'anàlisi.
- `int[] variableType` → array amb el tipus de pregunta de cada columna.

Valor de sortida:

- `double[]` → retorna un vector on les posicions que corresponen a columnes numèriques es guarda el màxim de la columna, altrament es guarda un -1.

6. `calculateMax95P`: calcula el valor del percentil 95 de la llista.

Paràmetres d'entrada:

- `List <Double> list` → llista dels valors.

Valor de sortida:

- `Double` → retorna el valor del nombre que correspon al percentil 95 de tots els de la llista.

Question.java -> Abstracta

1. `Question`: crea una instància de pregunta amb el seu enunciat i tipus corresponent. Aquests són els dos únics elements que tenen totes les classes concretes derivades d'aquesta, que és `ABSTRACTE`.

Paràmetres d'entrada:

- `String enunciat`: Enunciat de la pregunta.
- `Int tipus`: aquest enter ens indica quin tipus de pregunta és:
 - 1 = Resposta única (`SingleChoiceQuestion`)
 - 2 = Resposta múltiple (`MultipleChoiceQuestion`)

3 = Text lliure (FreeTextQuestion)

4 = Número lliure (FreeNumberQuestion)

2. crearQuestion: una vegada hem creat la instància de pregunta, hem de dir de quin tipu és. La classe abstracta no és una classe com a tal per ella sola, és **complete disjoint**. Aquesta classe fa que en el nostre codi podem crear una Question q i igualar-la a aquest mètode, que retorna una instància de la subclasse triada.

Aquesta classe té 3 funcions abstractes que es sobreescrueixen/ es fa override en cada subclasse: validarResposta, toString i ModifyQuestion. L'única que no està buida és toString que sempre retorna l'enunciat i el tipus de pregunta (valors de la classe pare i que, per tant, tenen les classes filles). Aquesta funció retorna una String. És bàsicament per interacció de qualitat amb l'usuari si executa el programa desde terminal i per debugació.

SingleChoiceQuestion

1. SingleChoiceQuestion: constructora de pregunta d'única resposta.

Paràmetres d'entrada:

- String enunciat: Enunciat de la pregunta.
- List<String> opcions: llista de respostes que té la pregunta

Sortida:

- S'ha creat una pregunta amb enunciat i diverses opcions de resposta. A més, es posa tipus = 1.

2. validarResposta: comprova que la resposta sigui vàlida. Quan responem una pregunta mirem si la resposta és realment un índex enter igual o més gran a zero i que es doni un índex menor al total d'opcions.

Paràmetres d'entrada:

- Object resposta: la resposta que rebem. Verifiquem que sigui de tipus int.

Paràmetres de sortida:

- Bool → True si la resposta a la pregunta és vàlida. Fals si no ho és.

3. toString: En el cas de pregunta de resposta única retorna les opcions que es poden respondre, a més del propi enunciat i el tipus, que agafa de toString de la classe pare.
4. ModifyQuestion: Imprimeix les opcions actuals i deixa posar-ne de noves, tal que es borren totes les anteriors i queden com a opcions les noves introduïdes separades per “;”.

MultipleChoiceQuestion

Aquest tipus de pregunta consisteix en una pregunta amb diverses respostes, de les quals es pot triar un cert nombre de respostes, o totes. Per això té un int maxRespostes que indica el màxim de respostes que podem triar. Si aquest valor és -1 vol dir que podem triar-les totes.

1. MultipleChoiceQuestion: igual que la constructora de la SingleChoiceQuestion, però amb un enter que indica el màxim de respostes que podem triar.
Paràmetres d'entrada: String enunciat, List<String> opcions, int maxRespostes.
2. validarResposta:
Paràmetres d'entrada: iguals que per SingleChoiceQuestion. Object resposta.
Paràmetres de sortida:
Bool → True si el paràmetre d'entrada és un vector d'enters. False si la llargada del vector d'índexs de respostes triades és més llarg que maxRespostes.
3. toString: igual que SingleChoiceQuestion, però la string retornada al principi indica el màxim de respostes que es pot triar.
4. ModifyQuestion: igual que SingleChoiceQuestion, però afegint que també es pot canviar el número màxim de respostes.

FreeTextQuestion

Aquest tipus de classe no té altres atributs que no tingui la pròpia classe Question. Es tracta d'un enunciat i un tipus de pregunta.

1. FreeTextQuestion: agafa la constructora de la superclasse i hi posa el número 3 que correspon a aquest tipu de pregunta.
2. validarResposta:
Paràmetres d'entrada: igual en totes, Object resposta.
Paràmetre de sortida: Bool -> True si tenim una cadena de caràcters de mida igual o menor a 100. També s'hi val posar un número com a resposta en aquesta subclasse.
3. ModifyQuestion: Només canvia l'enunciat ja que no conté respostes a triar.

FreeNumberQuestion

Molt semblant a la FreeTextQuestion.

1. FreeNumberQuestion: agafa la constructora de la superclasse i hi posa el número 4 que correspon a aquest tipu de pregunta.

2. validarResposta:
Paràmetres d'entrada: igual en totes, Object resposta.
Paràmetre de sortida: Bool -> True si entra com a resposta un enter o decimal.
3. ModifyQuestion: Només canvia l'enunciat ja que no conté respostes a triar.

Survey.java

1. Survey: Crea una instància de la classe Survey sense inicialitzar
Paràmetres d'entrada: cap
Valor de sortida:
 - La nova instància no inicialitzada de Survey
2. Survey: Crea una instància inicialitzada de Survey
Paràmetres d'entrada:
 - Id: identificador de l'enquesta
 - Descripcio: descripció de l'enquesta
 - Titol: titol de l'enquestaValor de sortida: la nova instància de survey inicialitzada
3. CanviTitol: Modifica el titol de l'enquesta del antic a un nou, insertat per terminal
Paràmetres d'entrada: cap
Valor de sortida: cap(void)
4. CanviDescr: Modifica la descripció de l'enquesta de la antiga a una nova, instertada per terminal.
Paràmetres d'entrada: cap
Valor de sortida: cap(void)
5. gestionarPreg: Permet modificar, esborrar o ignorar cada una de les preguntes de l'enquesta.
Paràmetres d'entrada: cap
Valor de sortida: cap(void)
6. afegirPreg: Permet decidir si volem afegir una nova pregunta i crear-la
Paràmetres d'entrada: cap
Valor de sortida: cap(void)
7. IntToObject: Converteix un array de int a un array de Object.
Paràmetre d'entrada:
 - Array de int

Valor de sortida: Array de Object

8. ModifySurvey: Permet decidir si modificar el títol o la descripció, afegir noves preguntes i, si hi han preguntes ja existents modificar-les o esborrar-les.

Paràmetres d'entrada: cap

Valor de sortida: cap

9. SurveyAnalysis: Analitza l'enquesta i agrupa les seves respostes en clusters utilitzant kmeans

Paràmetres d'entrada: cap

Valor de sortida: cap

10. SurveyAnswer: Crea una nova instància de Answer i la inicialitza amb les respostes introduïdes per terminal.

Paràmetres d'entrada: cap

Valor de sortida: cap

Analysis.java

1. Analysis: Creadora sense inicialitzar de la classe.

Paràmetres d'entrada: cap

Valor de sortida: nova instància de la classe Analysis.

2. Initialize: inicialitza els clusters de la classe

Paràmetres d'entrada:

- Int[] clusters: valors dels clusters

Valor de sortida: cap

3. Silhouette: Examina la qualitat dels clusters de diferents instàncies de kmeans i selecciona el millor.

Paràmetres d'entrada:

- Int [] result: resultat del kmeans
- Object[][] punts: posició dels punts sobre els quals es calcula kmeans
- Int [] type: array amb els tipus de pregunta
- Double[] min: array amb els valor mínims
- Double [] max: array amb els valors màxims
- Int k: k trada pel k-means

App.java

Atributs:

- List<Survey> surveyList - Llista que emmagatzema totes les enquestes disponibles

Mètodes:

- constructors:
 - App() - Constructor buit que inicialitza la llista d'enquestes
- inicialització:
 - initializeApp(Survey[] surveys) - Inicialitza l'aplicació amb un array d'enquestes

Paràmetres d'entrada:

 - Survey[] surveys: llista d'enquestes

Valor de sortida: cap
- visualització:
 - displaySurveyList() - Mostra la llista d'enquestes disponibles per consola
 - showMenu() - Mostra el menú principal de l'aplicació
 - showMenuSurveyManagement() - Mostra el menú de gestió d'enquestes
- interacció amb l'usuari:
 - interactWithUser() - Gestiona la interacció principal amb l'usuari
 - interactWithUserManagement() - Gestiona la interacció del menú de gestió
 - SurveyManagement() - Crida als mètodes de gestió d'enquestes
- gestió d'enquestes:
 - SurveyCreation() - Crea una nova enquesta
 - SurveyModification() - Crida a ModifySurvey() de la classe Survey per modificar una enquesta existent
 - SurveyDeletion() - Elimina una enquesta
- funcionalitats principals:
 - SurveyAnswer() - Crida a SurveyAnswer() de la classe Survey per respondre una enquesta
 - SurveyAnalysis() - Crida a SurveyAnalysis() de la classe Survey per fer l'anàlisi d'una enquesta utilitzant K-means
 - selectSurvey() - Permet seleccionar una enquesta de la llista

Answer.java

Atributs:

- Object[] answer - Array que emmagatzema les respostes d'un usuari a totes les preguntes

- `BufferedReader reader` - Objecte per llegir l'entrada de l'usuari des de la consola

Mètodes:

1. Constructors:

- `Answer()` - Constructor buit que inicialitza un array de respostes buit i el `BufferedReader`

2. inicialització i entrada de dades:

- `initialize(int[] type, ArrayList<Question> questions)` - Inicialitza les respostes demanant a l'usuari que respongui cada pregunta. Mostra cada pregunta i gestiona l'entrada segons el tipus.

Paràmetres d'entrada:

- `int[] type`: llista de tipus
- `ArrayList<Question> questions`: array de preguntes

Valor de sortida: cap

3. Mètodes de lectura d'entrada (privats):

- `readSingleChoiceOrdered(Question question)` - Llegeix una resposta d'opció única ordenada (per índex)

Paràmetres d'entrada:

- `Question question`: objecte pregunta

Valor de sortida:

- `Object[] result` de mida 2, on `result[0]` és l'índex seleccionat i `result[1]` el total d'opcions, si la resposta és vàlida
- `Object[0]`, si la resposta no és vàlida

- `readSingleChoiceUnordered(Question question)` - Llegeix una resposta d'opció única no ordenada (per text)

Paràmetres d'entrada:

- `Question question`: objecte pregunta

Valor de sortida:

- `String`: text de resposta, si la resposta és vàlida
- `""`, si la resposta no és vàlida

- `readMultipleChoice(Question question)` - Llegeix respostes múltiples (per índexs separats per coma)

Paràmetres d'entrada:

- Question question: objecte pregunta

Valor de sortida:

- Set<Object> de mida variable, on cada Objecte és una resposta seleccionada per índex, si la resposta és vàlida
- HashSet<>(), si la resposta no és vàlida
- readFreeText() - Llegeix una resposta de text lliure

Paràmetres d'entrada:

- Question question: objecte pregunta

Valor de sortida:

- String: text de resposta
- readFreeNumber() - Llegeix una resposta numèrica

Paràmetres d'entrada:

- Question question: objecte pregunta

Valor de sortida:

- Object: num com a resposta, si la resposta és vàlida
- null, si la resposta no és vàlida

4. getters i setters:

- setAnswer(Object[] answer) - Estableix l'array de respostes donat com el propi de la classe

Paràmetres d'entrada:

- Object[] answer: llista de respostes a una enquesta

Valor de sortida: cap

- getArray() - Retorna l'array de respostes

Funcionalitat principal:

La classe Answer s'encarrega de:

- Capturar les respostes d'un usuari a través de la consola
- Validar que les respostes coincideixin amb el tipus de pregunta esperat
- Gestionar diferents formats d'entrada segons el tipus de pregunta
- Emmagatzemar les respostes en un array d'Object per suportar múltiples tipus de dades

Tipus de respostes suportats:

- Tipus 0 (FreeNumber): Números (Double)
- Tipus 1 (SingleChoiceOrdered): llista de dos objectes, result[0]=índex, result[1]=numopcions
- Tipus 2 (SingleChoiceUnordered): Text amb el valor exacte de l'opció
- Tipus 3 (MultipleChoice): Set d'objectes amb els índexs seleccionats
- Tipus 4 (FreeText): Text lliure (String)

DESCRIPCIÓ DE JOCS DE PROVES (DRIVERS)

KMeansDriver.java

Driver per executar proves del sistema de classificació mitjançant KMeans.

Objecte de la prova:

Aquest executable prova la integració final del procés de clustering mitjançant l'algorisme KMeans. Primer de tot extreu les dades d'un fitxer (a elecció de l'usuari) i el tipus de pregunta al que correspon cada dada. A continuació, es processen les dades d'aquest fitxer de manera que s'eliminen valors nuls i es modifiquen els outliers. Finalment, s'envien les dades a KMeans per a realitzar el clustering.

- Seleccionar l'enquesta per respondre (escollint l'arxiu a analitzar).
- Seleccionar la k (nombre de clusters en els que es divideixen les dades).
- Veure l'anàlisi (obrint el fitxer results.csv, on es guarda el clustering).

Altres elements integrats a la prova:

- ReadAnswers.java per llegir les dades del fitxer i fer el pre-processament.
- KMeans.java per a executar l'algoritme que fa el clustering.
- AuxiliarMethods.java per a utilitzar mètodes específics com ara càlcul de la moda o la mitjana.

Fitxers de dades necessaris:

- iris.csv: conté vectors de característiques numèriques, amb cada fila etiquetada amb el seu grup real entre iris-setosa, iris-versicolor, iris-virginica.
- loan-loader.csv: conté informació numèrica referent a dades de préstecs. Es fa servir per validar el funcionament de l'algorisme amb dades numèriques i de resposta única. A més, conté cada fila etiquetada amb el valor del cluster real.

Cada fila d'aquests fitxers representa un objecte que es converteix en un vector Objecte que després s'utilitza per l'algorisme.

Valors estudiats:

La prova utilitza les dades completes dels fitxers CSV:

- Caixa negra: es comprova la precisió del algorisme per retornar els resultats corresponents als reals (que coneixem gràcies a les etiquetes de cada fila) sense necessitat d'analitzar l'estructura interna del codi.
- Caixa blanca: es validen comportaments per a conjunts homogenis per comprovar que vectors similars acaben al mateix cluster.

L'objectiu és observar el comportament de KMeans en diferents dades i estudiar la coherència entre resultats esperats i resultats generats.

Efectes estudiats:

A nivell funcional, el driver gestiona:

- Interacció per consola per seleccionar el fitxer de dades.
- Lectura i preprocessament de les dades del fitxer.
- Execució del algorisme i obtenció de resultats (incloent la precisió d'aquests):
- Generació d'un fitxer results.csv amb la classificació final.

Operativa:

1. El programa mostra un llistat amb els fitxers sobre els quals es pot fer l'anàlisi.
2. L'usuari selecciona el fitxer que vol analitzar.
3. L'usuari selecciona el valor de la k que vol.
4. El programa llegeix el CSV i calcula els clusters amb KMeans.
5. Es mostra per consola la precisió de l'algorisme.
6. Es guarden a results.csv els resultats del clustering.

App.java

Aplicació principal per gestionar, respondre i analitzar enquestes mitjançant una interfície d'interacció basada en consola.

Objecte de la prova:

Aquest executable prova la integració completa del sistema de gestió d'enquestes, incloent:

- Tots els casos d'ús relacionats amb la gestió d'enquestes (crear, modificar...).
- Resposta d'enquestes (de diferents tipus).
- Anàlisi d'enquesta (triada per l'usuari).

Altres elements integrats a la prova:

L'aplicació reutilitza diverses funcionalitats que ja han estat provades de forma individual.

- Survey.java, que s'encarrega de gestionar preguntes, respostes i anàlisis.
- Question.java i les seves subclasses per a diferents tipus de preguntes.
- Answer.java per capturar les respostes d'usuari.

- KMeansDriver.java per executar l'algorisme amb enquestes ja existents (el seu funcionament s'ha especificat a dalt).

Aquest executable combina aquestes classes per a comprovar el funcionament conjunt.

Fitxers de dades necessaris:

L'aplicació bàsica no requereix fitxers externs, però sí que depèn de les dades introduïdes per cada usuari per consola. Així doncs, l'executable no necessita carregar dades externes.

Valors estudiats:

L'executable permet provar el sistema amb qualsevol escenari que l'usuari desitgi. No obstant, cal garantir que almenys una dada introduïda és diferent de null per tal de realitzar l'anàlisi. S'aplica principalment una perspectiva de caixa negra per provar la interacció (ja que l'usuari que executa el codi no té perquè conèixer com s'ha implementat aquests. No obstant, trobem alguns casos de caixa blanca (comprovació d'una selecció de respostes correctes o recorregut de les dades).

Efectes estudiats:

A nivell funcional, el driver gestiona:

- Navegació per menús textuais i lectura d'opcions via consola.
- Interaccions de selecció per part de l'usuari (escollir enquesta, opció...).
- Fluxos complets per gestionar, respondre i analitzar una enquesta.
- Integració amb el sistema d'algoritmes d'anàlisi.

Operativa:

1. L'aplicació mostra un menú principal, donant a l'usuari l'opció de triar entre gestionar, respondre, analitzar enquesta o sortir del programa.
2. En funció de la opció triada, l'usuari pot:
 - Crear una enquesta.
 - Modificar o eliminar una enquesta existent (seleccionant el seu index).
 - Afegir preguntes a l'enquesta.
 - Fer un anàlisi KMeans de l'enquesta (triant l'enquesta i la k).

DESCRIPCIÓ DE CASOS D'ÚS

Nom: Gestionar enquesta

- Actor: Usuari
- Comportament: S'obre una pantalla on permet a l'usuari triar entre crear una nova enquesta o modificar-ne una d'existent.
- Errors possibles i cursos alternatius: -

Nom: Respondre enquesta

- Actor: Usuari
- Comportament: S'obre una pantalla amb totes les enquestes existents.
- Errors possibles i cursos alternatius: si no hi ha enquestes existents salta un missatge d'error.

Nom: Fer anàlisi

- Actor: Usuari
- Comportament: S'obre una pantalla amb totes les enquestes existents.
- Errors possibles i cursos alternatius: si no hi ha enquestes existents salta un missatge d'error.

Nom: Crear enquesta

- Actor: Usuari
- Comportament: S'obre una nova enquesta amb una sola pregunta en blanc
- Errors possibles i cursos alternatius: -

Nom: Modificar enquesta

- Actor: Usuari
- Comportament: S'obre una pantalla amb totes les enquestes existents.
- Errors possibles i cursos alternatius: si no hi ha enquestes existents salta un missatge d'error.

Nom: Seleccionar enquesta editar

- Actor: Usuari
- Comportament: S'obre l'enquesta seleccionada per la primera pregunta i permet a l'usuari editar-la.
- Errors possibles i cursos alternatius: -

Nom: Crear pregunta.

- Actor: Usuari
- Comportament: Afegeix una nova pregunta en blanc a l'enquesta seleccionada
- Errors possibles i cursos alternatius: -

Nom: Eliminar pregunta

- Actor: Usuari

- Comportament: Elimina la pregunta seleccionada
- Errors possibles i cursos alternatius: -

Nom: Seleccionar enquesta respondre

- Actor: Usuari
- Comportament: S'obre l'enquesta seleccionada i permet a l'usuari respondre les seves preguntes.
- Errors possibles i cursos alternatius: -

Nom: Seleccionar enquesta analitzar

- Actor: Usuari
- Comportament: s'obre una pàgina que et permet triar la k i veure l'anàlisi.
- Errors possibles i cursos alternatius: L'enquesta seleccionada ha de tenir alguna resposta, sinó en té salta un missatge d'error.

Nom: Triar k

- Actor: Usuari
- Comportament: Es tria la k per a fer l'anàlisi.
- Errors possibles i cursos alternatius: Si la k triada és major al nombre de respostes salta un missatge d'error.

Nom: Veure anàlisi

- Actor: Usuari
- Comportament: Es mostra l'anàlisi de l'enquesta triada utilitzant l'algoritme k-means amb la k indicada.
- Errors possibles i cursos alternatius: Si la k no ha estat triada, es calcula la millor opció utilitzant elbow method.

Nom: Editar següent pregunta

- Actor: Usuari
- Comportament: S'obre la pregunta següent i permet a l'usuari editar-la.
- Errors possibles i cursos alternatius: Si no existeix cap pregunta no fa res.

Nom: Editar pregunta anterior

- Actor: Usuari
- Comportament: S'obre la pregunta anterior i permet a l'usuari editar-la.
- Errors possibles i cursos alternatius: Si no existeix una pregunta anterior no fa res.

Nom: Sortir

- Actor: Usuari
- Comportament: Es desa l'enquesta editada i es torna a la pàgina inicial.
- Errors possibles i cursos alternatius: Si hi ha preguntes en blanc o no acabades, aquestes es desen en blanc.

Nom: Contestar següent pregunta

- Actor: Usuari
- Comportament: S'obre la pregunta següent i permet a l'usuari contestar-la.
- Errors possibles i cursos alternatius: Si no existeix una pregunta següent no fa res.

Nom: Contestar pregunta anterior

- Actor: Usuari
- Comportament: S'obre la pregunta anterior i permet contestar-la.
- Errors possibles i cursos alternatius: Si no existeix una pregunta anterior no fa res.

Nom: Acabar

- Actor: Usuari
- Comportament: Es guarda la nova resposta i es torna a la pàgina principal.
- Errors possibles i cursos alternatius: Si no s'han respost totes les preguntes aquestes es desen en blanc.

DIAGRAMA CASOS DÚS

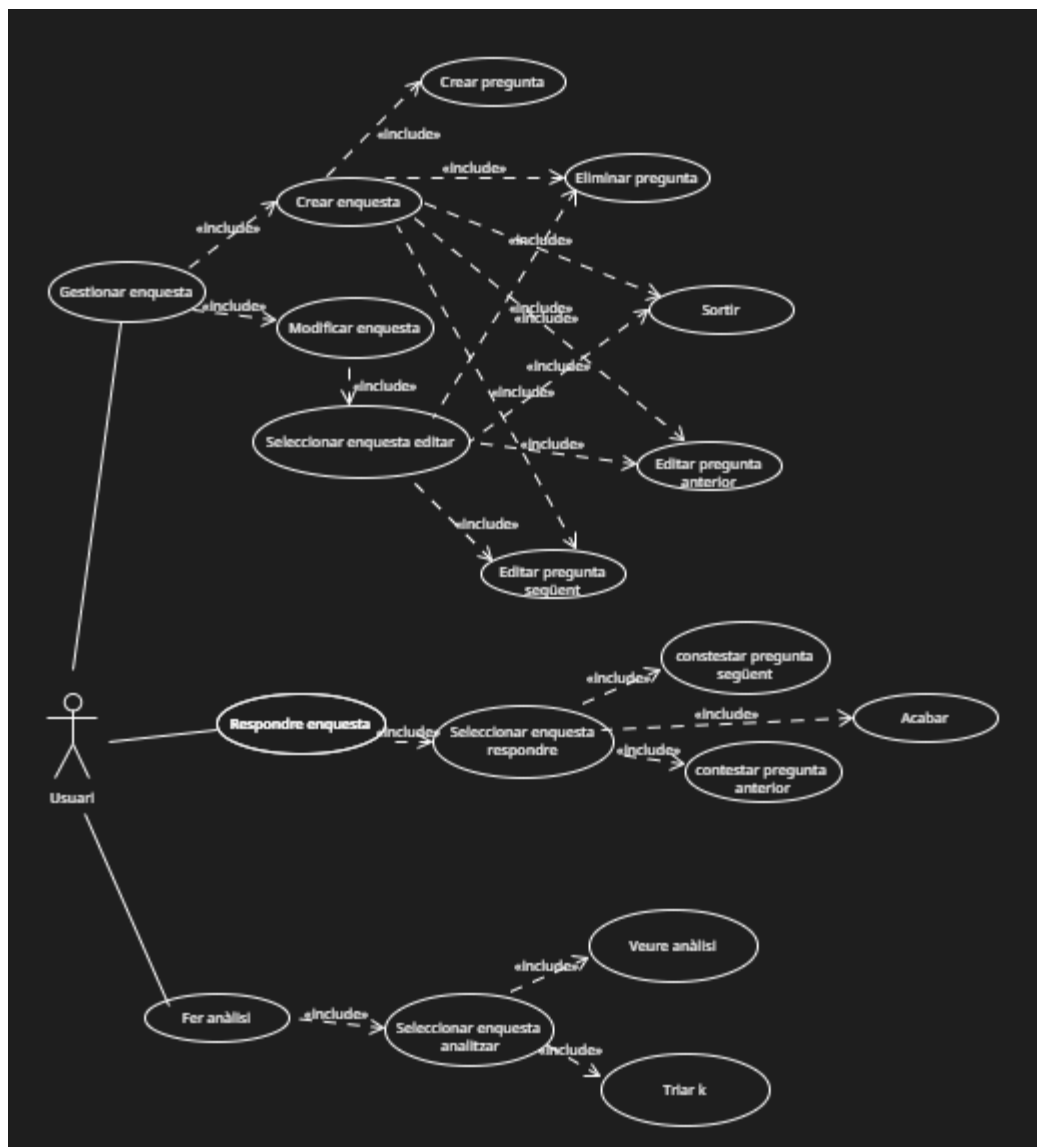
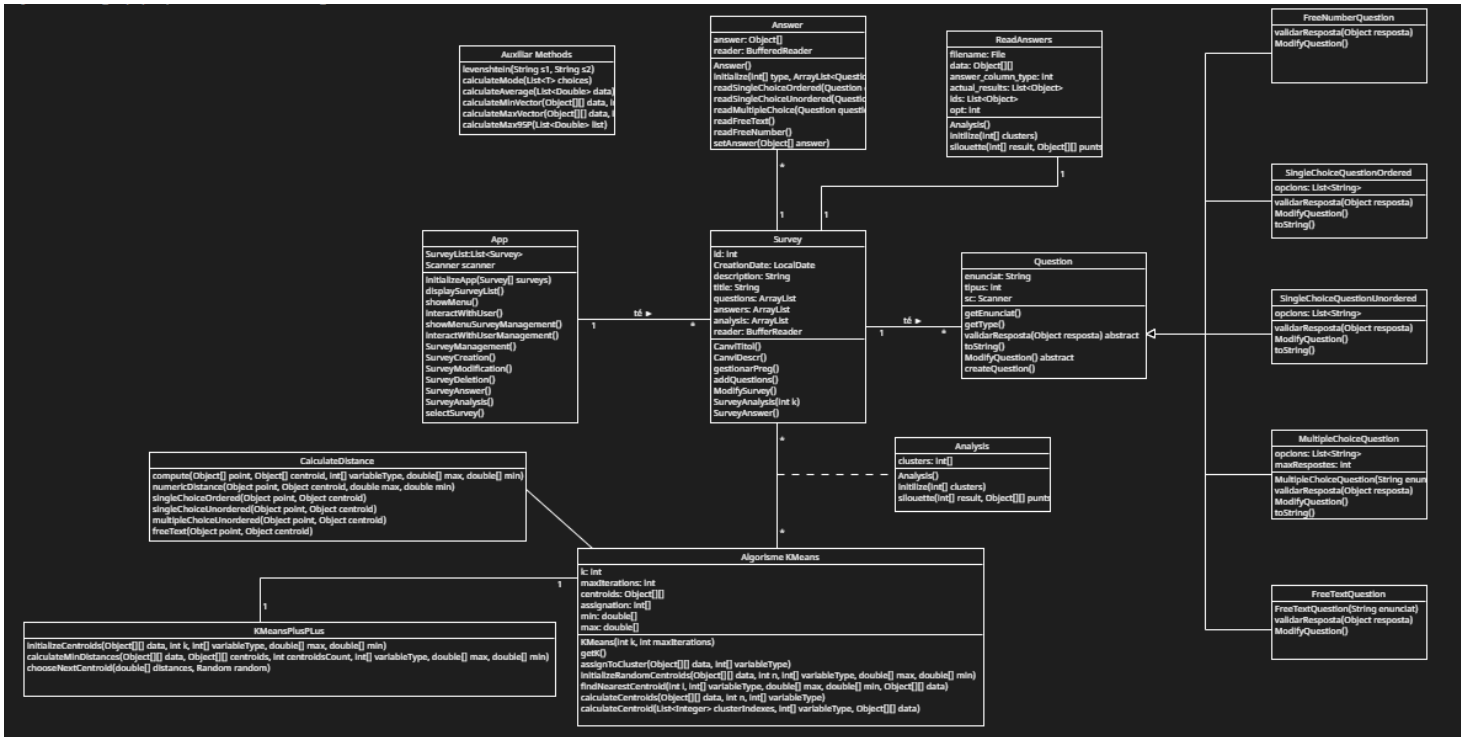


DIAGRAMA DE CLASES



CLASSES IMPLEMENTADES PER CADASCÚ

Paula → CalculateDistance, AuxiliarMethods, KMeans, KMeansPlusPlus, TestAccuracy, KMeansDriver, CalculateDistancesTest, AuxiliarMethodsTest

Abel → ReadAnswers, Question, Main, Makefile, QuestionTest, SurveyTest

Judit → Survey, Analysis

Aura → Answer, App, AppTest, AnswerTest